Brightloop — Independent Engineering Jury Evaluation

Overall score: 76 / 100 Verdict: Strong functional prototype; not yet a dependable production billing system.

Evaluated on 31 October 2026 against the user's attached four-page Subscription Management System problem statement (PS). This is an evaluator-designed rubric, not an official competition score or a prediction of another jury's marks.

1. Scope, provenance and confidence

• Repository: https://github.com/divyanshuwu/Subscription-Management-System

• Evaluated commit: `f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7`.

• Commit message: “initial prototype codebase, need testing and cleanup”.

• The attached PS and the repository's PDF have identical SHA-256: `80a2976f4c2ad05316975d61de32980ce622870c8d70748a37e42f1f992cf42a`.

• The attached PDF is the source of requirements. `PS_Module_Breakdown.md` is the project's own interpretation, not an official marking scheme. Its additional requirements were not silently treated as mandatory.

• Method: source inspection, clean dependency installation, production build, lint, integration tests, both browser suites, supplied API load test, independent isolated edge-case probes, overlapping billing-job test and dependency audit.

• Runtime used for executable backend checks: Node **24.22.2**, matching the README's Node 24 prerequisite; SQLite. Browser: headless Playwright Chromium.

• An initial run under the sandbox's Node 20 crashed in native SQLite before tests executed. After installing Node 24 and rebuilding the native dependency, all supplied backend tests passed. This environment mismatch is **not counted as a project defect**. Chromium system libraries also had to be installed before browser tests could execute.

• No business source was changed. Tests regenerated result files and local artifacts; independent probes use isolated in-memory databases. The original committed source snapshot is supplied separately.

Confidence: high for the tested SQLite happy paths and reproduced scenarios; moderate for overall requirement completeness; limited for PostgreSQL, real email delivery, production security, multi-instance deployment and the PS's full page-load performance guarantee.

2. Executive assessment

This is substantially more than a static dashboard. Brightloop has a working Express/Knex backend, persistent storage, JWT access and rotating refresh tokens, server-enforced roles and customer ownership, catalog and plan administration, reusable quotation templates, subscription transitions, scheduled/manual billing, tax/discount calculation, invoice PDFs, payment recording, reports, audit history and an API-connected customer UI.

The team deserves credit for implementing the complete business chain and for clearly distinguishing its browser-only demo from its authenticated local application. The normal flow from creating a subscription to recording payment is executable and tested.

The limiting factor is **financial correctness over time**, not missing screens. A billing system must preserve previously earned charges, honor a coherent pricing policy, expose all outstanding debt and avoid misleading finance totals. Independent probes found failures or incomplete policies in those areas. Real delivery of reset

links and invoices is also absent: the current delivery mechanism is a local database/file outbox.

Jury recommendation: award solid credit for breadth, runnable implementation and presentation, but withhold top-tier marks until lifecycle/billing correctness, date semantics, quotation consistency and delivery are fixed. Suitable for a prototype demonstration and controlled evaluation; do not put real customer billing on this version without remediation.

3. Scoring method and breakdown

Functional PS alignment carries 80 points; engineering execution carries 20. Full marks require an end-to-end usable feature with evidence of correctness, not merely a model, route or screenshot. Partial credit recognizes implemented infrastructure when delivery, policy or edge cases are incomplete. A finding is assigned primarily to its owning category rather than repeatedly deducted everywhere. Scores are professional judgment, not an automated coverage percentage.

Category	Maximum	Awarded	Reason for the score
Account creation, authentication and role access	12	10	Correct password rule, unique normalized email, RBAC, admin-only staff creation, reset-token security. Reset is a pasted token in a local outbox, not an emailed reset link.
Products, variants and configurable plans	13	12	Product fields, Odoo-style price delta, four frequencies, availability, minimum quantity and action flags exist. UI subscription creation is single-product; live add-on changes are not implemented.
Quotations and subscription lifecycle	13	9	Templates, validity, transactional single conversion, identifiers and core transitions work. Quote price is not preserved; direct Draft → Quotation is missing; ongoing subscription editing is limited.
Recurring billing, invoices, taxes and discounts	20	14	Transactional generation, unique periods, backfill, scheduler, invoice actions, PDF and rule engine are real. Pause/resume can skip debt; confirmation reprices discounts; sending is simulated/local.
Payments and receivables	9	8	Methods, dates, notes, partial payments, paid state and overpayment prevention work. Sub-cent input can produce a zero payment; financial state interpretation needs tightening.
Customer self-service portal	6	5	Owned lists/PDFs and permitted lifecycle actions work; server isolation tested. Old active records can disappear under default filters; editable add-ons are absent.
Finance reporting and the performance requirement	7	5	Live cash receipts, summary cards, filters and pagination; supplied 10k API benchmark passes. Debt aging and loaded-subset widgets can mislead; every page under two seconds remains unproven.
Architecture and maintainability	5	3	Sensible service/auth/database split and shared schemas. Very large frontend component, monolithic route file, unversioned migrations and duplicated demo/API business logic.
Testing and quality gates	6	4	15 integration, 10 demo-browser and 7 API-browser checks pass; load tooling exists. Lint fails with 14 errors; missing critical edge cases, coverage metrics and CI.
Security and operational readiness	5	3	Hashing, validation, token rotation and ownership checks are meaningful. Dependency findings, localStorage tokens, no delivery retries/production deployment or PostgreSQL verification.
UX, documentation and reproducibility	4	3	Coherent responsive UI, working flows, helpful README, credentials and configuration. Advanced setup is dense; selectors/global search use limited cached records and some form choices are ignored.
Total	**100**	**76**	**Functional: 63/80. Engineering: 13/20.**

This does **not** mean “76% production-ready.” Reliability-sensitive defects can block a production launch even when most features work.

4. PS-to-implementation traceability

Status vocabulary: **Met** = substantially implemented with inspection/test evidence; **Partial** = a meaningful part exists but a user outcome or important condition is incomplete; **Unproven** = implementation/claims exist but the complete requirement has not been established. “Met” does not imply exhaustive verification of every input.

PS reference	Required outcome	Status	Evidence and qualification
p2: signup	Email/password registration; one account per email	Met	Auth routes normalize email; DB uniqueness; duplicate-case integration test.
p2: password	More than 8 characters, uppercase, lowercase, special character	Met	Zod minimum 9 plus three regex rules; weak-password test passes.
p2: forgotten password	Reset link arrives at registered email	Partial	Secure expiring single-use token exists, but written to local outbox; no actual email or clickable reset link.
p2: roles	Admin, limited staff, customer portal	Met	Backend middleware, restricted catalog writes, customer ownership and authenticated UI.
p2: team	Only admin adds staff	Met	Customer/staff 403, admin 201 verified.
p3: products	Name, type, sales price, cost price, notes	Met	Catalog schemas, DB and advanced editor.
p3: variants	Variant selection changes price, e.g. Odoo +560	Met	Variant delta stored and applied by server pricing; duplicate group selections rejected in code.
p3: plans	Price, daily/weekly/monthly/yearly, minimum quantity, availability	Met	All four frequencies and validations implemented; not every frequency boundary has an independent automated test.
p3: plan policy	Automatic closure; early close, pause and renew configured per plan	Partial	Flags and transitions exist. Resume overwrites overdue billing date; end/renewal semantics need stronger policy tests.
p3: quotation templates	Plan, product lines and validity days	Met	Template CRUD; quotes generated with valid-until; expired conversion rejected.
p3: quote → subscription	Convert an offer into a subscription	Partial	Atomic and one-time, but conversion re-derives price from current catalog rather than preserving offer terms.
p3: subscription identity	Number, customer, plan, dates, payment terms	Met	Number sequences, schema and creation service. API supports expiry; simple UI chooses a one-period default and does not expose a custom expiry.
p3: subscription lines	Product, quantity, unit price, taxes and amount	Partial	Lines store product, quantity, effective price; invoice calculator adds tax/amount. Subscription lines/detail do not fully expose the required tax/amount snapshot.
p3: lifecycle	Draft → Quotation → Confirmed → Active → Closed	Partial	Converted quotes start as Quotation, direct subscriptions as Draft. No action moves an existing Draft to Quotation; other major transitions work.
p3: team visibility	Team can see subscription status	Met	Staff-accessible paginated lists, status badges and per-subscription history.

PS reference	Required outcome	Status	Evidence and qualification
p3: billing day	Generate hundreds of invoices from subscriptions	Met	Real generator; independent overlapping jobs produced exactly 500 invoices from 500 due subscriptions. SQLite-only evidence.
p3: invoice content	Customer, lines, taxes and totals filled in	Met	Customer snapshot, calculated lines/totals and locally generated PDF.
p3: invoice actions	Confirm, cancel, send, print	Partial	Confirm/cancel/download PDF work. Print is via PDF; send records a local message referencing a local PDF, not delivered mail/attachment.
p3: invoice state	Draft → Confirmed → Paid	Met	Confirmation and exact/partial payment behavior verified; zero-total confirmation appropriately becomes Paid.
p3: billing integrity	Nothing billed twice or skipped	Partial	Unique subscription/period constraint and repeat/concurrent SQLite checks pass. Resume can discard a previously due period.
p3: taxes	Automatic admin-defined rules	Met	Percent/fixed rules and product scopes implemented. Tax jurisdiction/rounding policy is not specified by PS and still needs a documented production decision.
p4: discounts	Admin-only; fixed/percent; minima, dates and usage cap	Met	Schemas/RBAC/calculator/confirmation accounting; expired and exhausted rules skipped in tests.
p4: discount application	Product/subscription scope, automatic next invoice, expires/runs out	Partial	Automatic evaluation exists. Draft and confirmation use different effective dates/current rules, allowing an unexpected total change.
p4: payments	Method, amount, date, note, correct invoice	Met	Persisted fields, staff recording, FK ownership, partial payment sum and negative/future/overpayment validation. Sub-cent defect remains.
p4: visibility	Paid, outstanding and overdue invoices	Partial	Views exist; drafts treated as receivables/overdue; old debt omitted by default aging range.
p4: finance	Active count, receipts, owed amount, overdue invoices, filters, summary	Partial	Core totals and live revenue chart exist. Some overview widgets/global search operate on only loaded records; historical summary backfill/reconciliation is weak.
p4: speed	Every page under two seconds with thousands of subscriptions	Unproven	13 API endpoints pass a 10k-subscription benchmark. No full-page p95/LCP proof across all pages, large finance ledgers or PostgreSQL.
p1–2: context	Unified workspace and customer self-management	Partial	Unified backend/UI and lifecycle self-service exist. Add-ons can be present in API lines/templates, but no live line/add-on edit endpoint/workflow was found.

5. What was actually executed

Check	Observed result	Interpretation
`npm ci --no-audit --no-fund`	Completed	Native dependency rebuilt after selecting supported Node 24.
`npm run build`	Pass	TypeScript + Vite production output builds successfully. Imported frontend files participate through the root entry point.
`npm test` on Node 24	**15/15 pass**	REST/database integration checks; about 2.1 seconds here.
`npm run test:ui`	**10/10 pass**	Browser-only demo, navigation, lifecycle, PDF and 390px layout; no reported browser errors. Not proof of backend security.
`npm run test:local-ui`	**7/7 pass**	Actual JWT login, SQLite-backed writes, state changes, billing, payment, PDF, product edit and customer login.
`npm run lint`	**Fail: 14 errors**	Unused imports, explicit any, hooks/purity checks, ignore comment and empty catch. Backend `.mjs` files are outside the configured TS/TSX rule block.
Supplied `npm run test:load`	**13/13 endpoint p95 checks pass**	10k subscriptions, 50 requests/endpoint, concurrency 5, 50-row pages, warm local SQLite API.
Independent overlapping billing test	**500 invoices, zero duplicate periods, zero failed subscriptions**	Two runs launched together; combined wall time **692ms** in isolated in-memory SQLite. Single-connection serialization limits generalization.
Independent diagnostic probes	**10 behaviors reproduced**	Includes eight concrete gaps/defects and two policy/resilience caveats; these are not ten equivalent security vulnerabilities.
`npm audit`	**15 affected-package findings: 12 moderate, 3 high**	No critical findings. Production-only audit reports 3 high affected packages. Audit is not an exploitability assessment.

API performance measurements

Endpoint	p95 milliseconds
`/subscriptions`	9.92
`/subscriptions?page=100`	13.13
`/subscriptions?status=Active`	39.99
`/invoices`	8.39
`/invoices/outstanding`	9.59
`/payments`	6.17
`/dashboard/summary`	4.07
`/reports/revenue`	4.13
`/reports/aging`	4.88
`/reports/subscriptions`	14.60
`/reports/invoices`	7.86
`/reports/payments`	3.65
`/audit`	9.47

The result is positive but narrower than the PS. The load seed expands subscriptions to 10,000 while leaving only the small demonstration invoice/payment ledger (28 seeded invoices), and the added subscriptions are next due in the future. This is not a 10,000-customer monthly billing simulation, an invoice-history stress test, or an end-to-end browser page-load benchmark. Cached daily totals also avoid the cost of recomputing the whole ledger on reads. Do not claim “every page under two seconds” from these measurements alone.

6. Reproduced findings and recommended corrections

F1 — Previously due billing can be skipped on pause/resume

Severity: High business correctness. PS p3 explicitly says nothing should be skipped.

Probe P6: an active subscription already due on **2026-09-30** was paused and resumed on **2026-10-31**, without running billing in between. Resume replaced its due date with today. Billing then generated only the 2026-10-31 period; the previously due period was not invoiced.

Source: `backend/app.mjs:640–641` sets `next_billing_date = today()` whenever resume sees an old date.

Fix: preserve pre-pause accrued service separately from the paused interval. Record pause/resume events and explicitly decide which service intervals are billable. Never erase accrued periods merely by advancing the cursor. Add a regression test for overdue-before-pause, long pause, renewal and resume near expiry.

F2 — A quotation's offer price is not preserved

Severity: High commercial integrity; inferred from the quote workflow, not an explicit immutable-pricing sentence in the PS.

Probe P2: template line price 100; create a valid quote; change the plan price to 200; convert quote. The resulting subscription line price is **200**, despite the quote remaining valid.

Source: `backend/app.mjs:417–461` (`validatedLines`) recomputes prices; quote creation/conversion at 516–583 does not establish a frozen accepted-price contract. `unit_price` is accepted by the line schema but ignored by the pricing validator.

Fix: snapshot calculated offer lines, prices, selected variant deltas and relevant terms at quote creation. Conversion should copy the valid quote's agreed terms or require an explicit, audited re-quote. Reject or explain unsupported price overrides rather than silently ignoring them.

F3 — Confirming an invoice changes its discount/total

Severity: High invoice consistency; effective-date policy is unspecified by PS but must be coherent.

Probe P3: backfill a service period starting **2026-09-30**, eligible for a then-valid 10% discount and a 10% tax. Draft total is **100**. Confirm on **2026-10-31**, after the discount expires: confirmed total becomes **110**.

Source: generation calculates with service `period_start`; `backend/services.mjs:314` recalculates with `today()` and current tax/discount rules. The same mechanism can alter taxes after an administrator edits a rule. Usage caps are correctly guarded at confirmation, but that does not resolve date/snapshot semantics.

Fix: define the invoice pricing effective date, version/snapshot the evaluated rules and preserve it through confirmation. If capacity changes require recalculation, show the difference and require explicit approval. Test expiry between generation and confirmation and parallel use of the final discount allowance.

F4 — “All” lists hide older still-relevant records

Severity: High operational visibility.

Probe P1: a still-active subscription created two years ago exists and is accessible by ID, but the default subscription list returns **zero records** for it. The shared list helper applies a rolling one-year `created_at` filter not just to reports, but also catalog, customers and active subscriptions.

Source: `backend/app.mjs:123–150`; frontend server lists also initialize a one-year range.

Fix: operational lists should default to all relevant records, with pagination and indexes. Use date ranges as explicit filters; for service views use service dates, and for debt views use outstanding state. Long-lived customers/products must not disappear because they were created earlier.

F5 — Receivable/aging views can misrepresent debt

Severity: High finance/reporting concern; draft recognition is a policy gap rather than an explicit PS state rule.

Probe P4: a **Draft** invoice contributes 100 to outstanding and appears overdue. Notifications use the same not-Paid/not-Cancelled selection, so an unconfirmed draft can receive an overdue notice.

Probe P5: a confirmed, unpaid invoice due **2024-10-31** appears in overall outstanding as **100**, but default aging buckets sum to **0**, because aging filters due dates to the last year.

Source: `backend/services.mjs:127–141`; `backend/app.mjs:882–903`; reminders in `backend/services.mjs:413–440`.

Fix: distinguish unissued drafts from collectible invoices, use an explicit aging “as of” date, include all open debt before that date and reconcile bucket totals to the same receivables population. Allow date-scoped reports without presenting them as all debt.

F6 — The stated subscription state sequence is incomplete

Severity: Medium PS mismatch.

Probe P7: POST an existing Draft subscription's `quotation` action → **404 Unknown action**. There is a separate template → quotation → subscription path, but a directly created Draft cannot traverse the stated Draft → Quotation sequence.

Source: `backend/app.mjs:592–606` action map.

Fix: add an explicit Draft → Quotation workflow and UI, or unify the quotation/subscription model with a documented interpretation accepted by the jury. Keep transition history and permissions. Also expose line taxes/amounts and a custom expiry/contract term in the normal subscription workflow.

F7 — Payment validation happens before money rounding

Severity: Medium data integrity.

Probe P8: submit amount **0.001**; the API accepts a positive amount, rounds it and creates a payment of **0** with HTTP 201.

Source: `backend/app.mjs:745–765`.

Fix: validate supported decimal precision and require the normalized amount to remain positive; use integer minor units or a decimal arithmetic library throughout money calculations. Reject non-cent values rather than silently changing them.

F8 — Real email delivery is not implemented

Severity: Medium-to-high requirement completion; not concealed by the README.

`email()` inserts a notification and appends plaintext to `backend/data/outbox.log`. Reset content asks the user to paste a token, and invoice send content references a local PDF path. No SMTP/provider delivery or real attachment handoff is present. This does not satisfy a reset link arriving in the registered mailbox.

Source: `backend/services.mjs:16–33`, `backend/auth.mjs:154–177`, invoice send action `backend/app.mjs:718–738`.

Fix: provide a configurable delivery adapter; send clickable expiring reset links and actual invoice attachments/authorized links. Use an outbox state machine with pending/sent/failed, attempts and retry scheduling. Current insert-before-file-write can suppress a retry if the file write fails after the unique notification key is committed. Do not log reset secrets in production.

F9 — Partial frontend datasets affect “whole business” experiences

Severity: Medium; source-inspected, not a separately scaled browser reproduction.

Staff refresh fetches only the first **100** rows of customers/plans/subscriptions/invoices/etc. ServerRecords correctly paginates its lists, but creation selectors and global search use cached subsets. `overdueAmount` is computed from loaded invoices rather than `serverSummary.overdue_amount`; upcoming renewals are also selected from the loaded subset. Some cards use server-wide totals correctly, so this is not a claim that all metrics are wrong.

Source: `frontend/src/BrightloopApp.tsx:391–429`, `531–550`, `3654–3684`.

Fix: server-side searchable selectors, dedicated upcoming-renewal endpoint, global server search and aggregate-backed widgets. Make selected-period versus all-time semantics visible. In the quotation form, the displayed plan selector does not control the API request, which sends only template/customer; remove it or synchronize it with the template's plan.

F10 — Quality and operational gaps

Severity: Medium overall; no demonstrated production compromise.

• Lint has 14 errors. The main frontend file is about **4,490 lines**, stylesheet about **4,119**, API route file about **995**. Size alone is not a defect, but mixed responsibilities and duplicate demo/API logic make finance changes hard to reason about.

• `migrate()` creates missing tables, but there are no versioned upgrade migrations. Existing deployments cannot evolve schemas safely through this mechanism alone.

• No committed CI workflow was found. Supplied integration tests share sequential state and miss the reproduced edge cases. There is no measured coverage report.

• Access/refresh tokens reside in localStorage. This increases impact of any future XSS; it is not evidence of a current XSS exploit. Password reset/logout revoke refresh tokens but existing access JWTs remain valid until their 15-minute expiry.

• Audit reports 3 high affected packages: `react-router`, `react-router-dom`, and `vite`; 12 moderate findings mainly in lint/tooling dependencies. They are not 15 independent remotely exploitable flaws. React Router server/RSC paths and dev-server advisories may not be reachable in this client-only/local setup. Inspect actual use and remove unused dependencies or upgrade compatible versions.

• Local binding/configuration is deliberate for a self-hosted demo, not automatically a security failure. However, production reverse proxy/TLS, secrets rotation, backups/restore, deployment packaging and monitoring are not demonstrated. Docker Compose covers PostgreSQL, not the entire application.

Fix: modularize by domain, enforce build/lint/test/audit in CI, add backend lint/type checks, introduce versioned migrations, upgrade/remove flagged dependencies, document token/session and deployment policies and test actual PostgreSQL transactions/concurrency.

7. Policy ambiguities and unverified risks — do not overstate them

1. **Billing anchor drift (P9):** Jan 31 → Feb 28 → Mar 28, and Feb 29 annual billing does not return to Feb 29 in the next leap year. The implementation clamps valid dates, so it is not a date crash. The PS does not define end-of-month anchoring; record an explicit anchor policy and test it. No separate mandatory-PS deduction solely for this behavior.

2. **Historical materialization gaps (P10):** an imported historical payment or a missed daily snapshot can be present in live `/analytics/revenue` but absent from `/reports/revenue`. The probe directly inserted historical data; ordinary payment POST refreshes its payment-date summary, so it is not proof that standard API-recorded backdated payments disappear. Add summary backfill/reconciliation and compute historical snapshots accurately.

3. **Partial-period end dates:** the generator calculates a full next period, not a prorated/truncated last period. The PS does not prescribe proration. Define policy before changing it.

4. **MRR/ARR versus collections:** current revenue charts represent cash receipts, not a normalized committed recurring revenue metric or recognized revenue. “How much revenue came in?” is substantially supported. MRR/ARR would improve business value, but is not an explicit named output in the PDF.

5. **No payment gateway/bank reconciliation:** payment recording is explicitly required; automated charging and bank-feed matching are not. Do not penalize absence of Stripe, Razorpay, microservices, Kubernetes or a cloud deployment as mandatory PS noncompliance.

6. **PostgreSQL:** configuration/schema support exists, but all executed backend checks used SQLite. PostgreSQL numeric serialization, locks, migration behavior, concurrency and performance remain unverified.

7. **Security evidence boundary:** sampled customer isolation tests passed; this is not a penetration test, exhaustive authorization proof or assurance against every attack.

8. Architecture review

Current flow: React/TypeScript UI → authenticated Express REST API → Zod/RBAC/domain services → Knex → SQLite or PostgreSQL. An in-process cron invokes billing/reminders; PDFKit writes local PDFs; notification rows and a log file stand in for mail delivery.

Good decisions worth retaining

• Server authority over roles and customer ownership; the demo role switch is not confused with actual access control.

• Unique `(subscription_id, period_start)` invariant in the database, per-invoice transactions and PostgreSQL row-lock branches.

• Refresh-token hashing/rotation, short access expiry, bcrypt and generic forgot-password response.

• Customer/pricing snapshots for invoices, audit entries alongside many business mutations and soft archive of core catalog data.

• Bounded page sizes, useful indexes, practical SQLite onboarding and clear local configuration.

• Real integration/browser tooling and transparent documentation of the demo/live split.

Recommended target, without unnecessary complexity

Keep the modular monolith. Split domain modules into identity, catalog, quotations, subscriptions, billing, payments and reporting. Centralize money/date policies. Introduce immutable quote/issued-invoice snapshots, a billable-period ledger, versioned migrations and a durable mail/job outbox. Use server-side aggregates and searchable selectors instead of broad frontend preload. Add a worker only when operational needs justify it; do not rebuild as microservices to solve bookkeeping defects.

9. Prioritized improvement plan and acceptance criteria

Effort estimates assume one engineer familiar with the stack; they are planning ranges, not delivery commitments.

Priority	Work	Rough effort	Acceptance criterion
P0	Preserve accrued service across pause/resume	1–3 days	Previously due periods billed exactly once across pause, resume, retry and renewal.
P0	Quote and invoice pricing snapshots/effective dates	2–4 days	Catalog/rule changes cannot silently change an accepted quote or an approved invoice; usage-cap races handled explicitly.
P0	Fix operational date defaults, aging and draft receivables	1–3 days	Old active records remain discoverable; all open debts reconcile across summary, lists and aging. Drafts are clearly excluded or separately labeled.
P1	Real reset/invoice mail and retryable outbox	1–3 days	Mailbox receives clickable single-use reset link and correct invoice; simulated delivery failure retries without duplicates.
P1	Complete Draft → Quotation, contract terms and line/add-on workflow	2–4 days	Full PS lifecycle demonstrated; custom expiry; visible tax/amount lines; changes take effect according to explicit billing policy.
P1	Money validation, precision and billing-anchor tests	1–2 days	Sub-cent values rejected; no zero/negative recorded payments; monthly/yearly boundaries tested.
P1	Server aggregates/searchable selectors	1–3 days	Customer 101+ selectable, search finds old/unloaded records, overdue and upcoming-renewal widgets agree with server queries.
P1	CI, lint cleanup, regression tests, dependency review	1–3 days	Clean build/lint/tests; new edge tests fail on old code and pass on fixes; advisory dispositions recorded.
P2	PostgreSQL, recovery and realistic performance proof	2–4 days	Same tests on PostgreSQL; restore rehearsal; overlapping billing jobs; browser timings with 10k subscriptions and large invoice/payment history.

Prioritize the P0 accounting issues above more charts, animation or new payment methods. Roughly a focused multi-week hardening effort is more valuable than another broad feature sprint; validate scope before committing to dates.

10. Suggested jury demonstration and challenge questions

Ten-minute evidence-led demo

1. Start from the README on Node 24; explicitly select authenticated API mode, not just the demo.

2. Register a customer; reject weak password and duplicate email; show staff cannot add staff or discounts.

3. Create a product with Odoo +560, a plan with flags and a reusable quotation template.

4. Create/convert a quotation once; show expired and duplicate conversion rejection.

5. Confirm/activate subscription; demonstrate customer ownership and plan-gated self-service.

6. Run billing twice; show invoice period uniqueness, correct totals and PDF.

7. Confirm invoice, record partial and final payments; show balance/paid state and reports.

8. Show the test/load evidence, then transparently disclose local email and the identified edge cases rather than claiming production readiness.

Questions that distinguish engineering depth

• What happens to already accrued debt when a subscription is paused before the billing job runs?

• What is the legally/commercially authoritative price: current plan, quotation snapshot or invoice snapshot?

• Which effective date governs discounts on a backfilled invoice, and why can confirmation change the total?

• How do “outstanding,” “overdue,” “draft” and “aging as of date” reconcile?

• Can a two-year-old active subscription and debt older than one year be found without changing filters?

• What is the end-of-month anchor and partial-period policy?

• How do two API instances coordinate billing, refresh rotation and notification delivery on PostgreSQL?

• How do customers/plans beyond the first 100 reach creation forms and global search?

• What exactly did the 10k load test measure, and what is still unmeasured about the under-two-second page requirement?

• What proof exists that reset links and invoices reach real mailboxes?

11. Reproduction and evidence inventory

The cloned repository is `/home/user/Subscription-Management-System`. Independent scripts and evidence are under `/home/user/evaluation`.

With Node 24 selected and dependencies installed in the repository:

```
cd /home/user/Subscription-Management-System
npm ci
npm rebuild better-sqlite3
npm run build
npm test
npx playwright install --with-deps chromium
npm run test:ui
npm run test:local-ui
npm run test:load
npm run lint
node /home/user/evaluation/jury-probes.mjs
node /home/user/evaluation/billing-batch.mjs
```

The independent scripts import the sibling clone and use in-memory SQLite. Diagnostic “REPRODUCED” means the script confirmed a described behavior, **not** that the behavior is desirable. P9 and P10 are specifically

caveated policy/resilience probes. Test commands regenerate result files/local outbox/PDFs; run on a disposable checkout, not against a real production database.

Evidence bundle includes build/lint/test logs, freshly generated browser and load results, 10 diagnostic outputs, overlapping billing output, npm audit JSON, extracted PS text/hash, original commit metadata and browser screenshots. The source ZIP is made from the evaluated commit, not the project's potentially stale bundled source download. No real email, payment processor or user production data was accessed.

12. Pinned source reference index

All links target the evaluated commit, not a moving default branch.

• Auth/password/reset: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/auth.mjs#L16-L208

• Shared list/date behavior: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/app.mjs#L123-L171

• Pricing and subscriptions: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/app.mjs#L417-L487

• Quotation/conversion: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/app.mjs#L516-L583

• Lifecycle/resume: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/app.mjs#L585-L651

• Invoice send/payment: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/app.mjs#L718-L811

• Finance reports: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/app.mjs#L841-L953

• Notification/calculator: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/services.mjs#L16-L117

• Summary and billing: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/services.mjs#L118-L301

• Invoice confirmation: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/services.mjs#L303-L353

• Billing-date function: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/db.mjs#L75-L89

• Invoice constraints: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/backend/db.mjs#L231-L253

• UI preloads/summary: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/frontend/src/BrightloopApp.tsx#L369-L550

• API request/token storage: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/frontend/src/data.ts#L455-L503

• Load-test dataset: https://github.com/divyanshuwu/Subscription-Management-System/blob/f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7/scripts/load_seed.mjs

Final jury statement: “Brightloop is a credible, well-presented full-stack prototype with substantial PS coverage and passing executable workflows. Its current ceiling is set by unhandled billing/reporting semantics and incomplete delivery, not lack of effort or screens. I award **76/100**, recommend a strong prototype assessment, and require the identified accounting and lifecycle corrections before production use.”